

Practical 3: Inputs and Outputs

This lab focuses on getting inputs from the user, performing some computations and displaying the outputs. We will build a simple mobile app to calculate the Body Mass Index (BMI). It is a handy tool for measuring overweight and obesity. It can be calculated with your weight and height. In a gist, BMI is a good indicator for diagnosing body fat percentage and potential diseases.

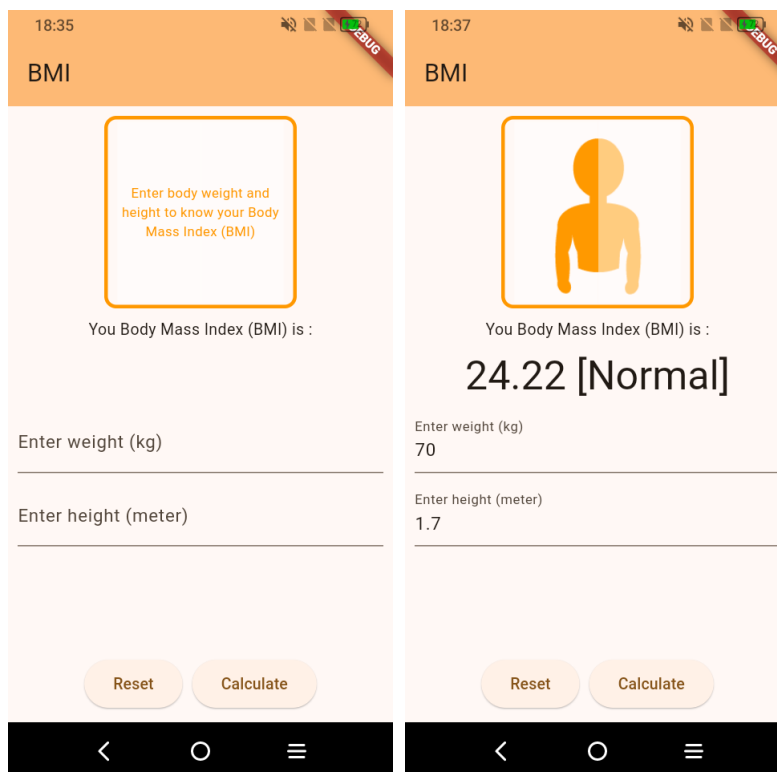
The formula for BMI calculation is as follows: -

$$\text{BMI} = \text{weight} / (\text{height in meters})^2$$

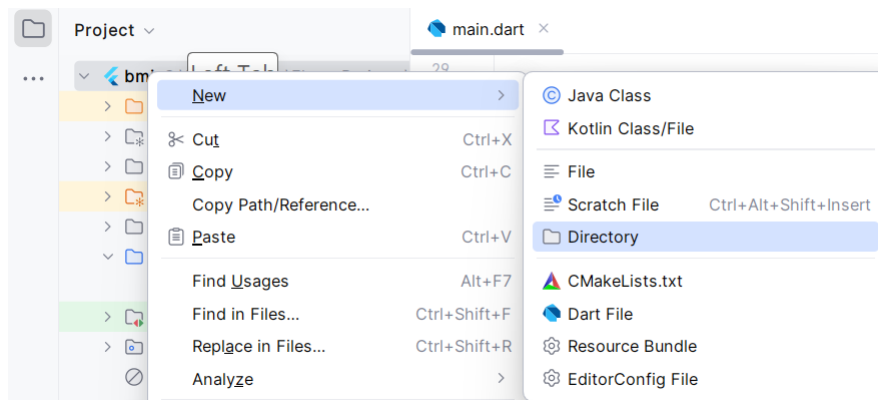
BMI values and types are as follows: -

BMI	Value
Underweight	Below 18.5
Normal	18.5 – 24.9
Overweight	More than 25

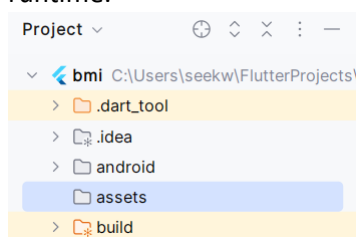
Sample outputs of the app are as follows: -



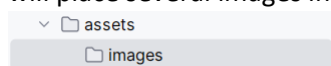
1. Create a new **Application** project. Set the project title to **bmi**.
2. In the project **File Explorer**, right click on the **root** of the folder and select **New -> Directory**.



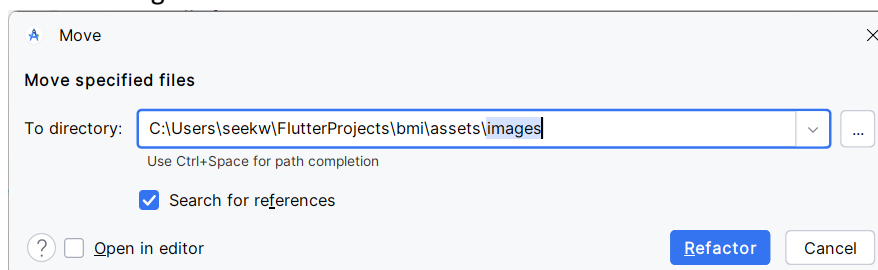
3. Name the new directory **assets**. The assets directory is dedicated for you to store static resources (images, audios, videos, fonts, JSONs, etc) that your app needs to function. These resources are then bundled with your app during the build process and become accessible at runtime.



4. Repeat step 2, within the assets directory, create a new sub-directory named **images**. We will place several images into this folder.

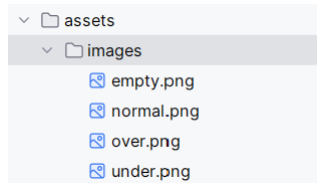


5. Copy and paste the four image files that represent BMI status into the **assets/images/** directory. When you paste the image files into the project File Explorer, the editor will prompt the following: -



Click the **Refactor** button to continue.

6. After that, the four image files should appear in the directory.



7. In the project File Explorer, open the **pubspec.yaml** file. Then scroll to the **flutter:** section. You are to insert a sub-section named **assets:**

```

53 flutter:
54
55   # The following line ensures that the Material Icons font is
56   # included with your application, so that you can use the icons in
57   # the material Icons class.
58   uses-material-design: true
59
60   #TODO 1 : Insert assets
61   assets:
62     - assets/images/
63   # To add assets to your application, add an assets section, like this:
64   # assets:
65   #   - images/a_dot_burr.jpeg
66   #   - images/a_dot_ham.jpeg

```

Below the **assets:** section, insert **– assets/images/**. This ensures the app can bundle all the images in this directory.

8. Open the **main.dart** file, at the beginning of the file, insert the following code:

```
import 'dart:math';
```

The import statement is to make code from external libraries accessible within your current file. In this case, we will use the power function (**pow**). Usually, the editor will insert unambiguous libraries automatically.

9. Within the class **_MyHomePageState**, we declare some local variables and **TextEditingController**s.

```

//TODO 2: Declare Local variable
double _bmi = 0.0;
double _weight = 0.0;
double _height = 0.0;

String _bmiOutput='';
String _bmiImage = 'assets/images/empty.png';

final TextEditingController _weightCtrl = TextEditingController();
final TextEditingController _heightCtrl = TextEditingController();

```

The **TextEditingController** is a class that manages the text input within a **TextField** widget. It can get from and set text to a **TextField**.

10. Insert the **_calculateBMI()** function to get user inputs and calculate the value of BMI.

```
//TODO 3: Calculate BMI value
void _calculateBMI() {

  _weight = double.tryParse(_weightCtrl.text)!;
  _height = double.tryParse(_heightCtrl.text)!;

  setState(() {
    _bmi = _weight / pow(_height, 2);

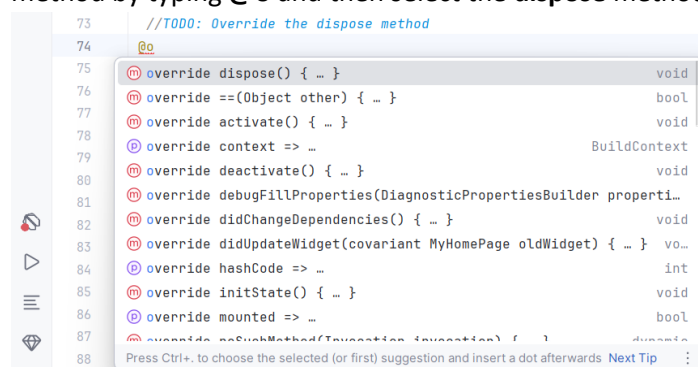
    if (_bmi < 18.5) {
      _bmiImage = 'assets/images/under.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Underweight]';
    } else if (_bmi >= 25) {
      _bmiImage = 'assets/images/over.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Overweight]';
    } else {
      _bmiImage = 'assets/images/normal.png';
      _bmiOutput = '${_bmi.toStringAsFixed(2)} [Normal]';
    }
  });
}
```

11. Next, insert a function named **_resetScreen** to reset state all widgets.

```
//TODO 4: Reset all widgets
void _resetScreen(){
  _weightCtrl.clear();
  _heightCtrl.clear();
  setState(() {
    _bmi = 0.0;
    _bmiOutput = '';
    _bmiImage = 'assets/images/empty.png';
  });
}
```

The `setState()` method is crucial for updating the UI in a stateful widget. When you call `setState()`, it tells Flutter that the widget's state has changed.

12. Within the **_MyHomePageState**, override the `dispose` method. You may override any class method by typing **@o** and then select the **dispose** method.



All **TextEditingController** should be disposed within the `dispose` method. The **dispose** method is called when the user is leaving your app. Controllers hold references to the text they manage. Dispose these controllers avoid memory leaks, which can prevent performance issues or even app crashes.

Complete the following codes:

```
//TODO 5: Override the dispose method
@override
void dispose() {
  super.dispose();
  _weightCtrl.dispose();
  _heightCtrl.dispose();
}
```

13. Within the Scaffold widget, insert a new attribute named **resizeToAvoidBottomInset** and set it to **false**.

```
return Scaffold(
  //TODO 6: set resize of screen to avoid overflow issue
  resizeToAvoidBottomInset: false,
  appBar: AppBar(
    backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    title: Text(widget.title),
  ),
```

14. Wrap the **Column** widget with a **Padding** widget, which adds empty space around its child. Within the **Padding** widget:

```
body: Center(
  //TODO 7: Insert a Padding widget
  child: Padding(
    padding: const EdgeInsets.all(8.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        ...
```

15. Within the children of the **Column** widget, insert a **Stack** widget.

```
//TODO 8: Insert Stack with image and text
Stack(
  fit: StackFit.loose,
  alignment: AlignmentDirectional.center,
  children: [
    //TODO 9: Insert two containers
  ],
),
```

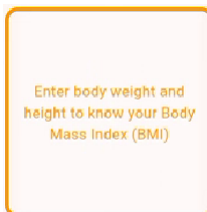
16. Within the Stack widget, insert two Container widgets. The first container holds an Image widget that represents the BMI status.

```
Container(
  width: 160,
  height: 160,
  decoration: BoxDecoration(
    border: Border.all(
      color: Colors.orange,
      width: 3,
    ),
    borderRadius: BorderRadius.circular(10.0),
  ),
  child: Image.asset(
    _bmiImage,
  ),
),
```

The second container holds a **Text** widget that displays instructions guiding users on how to operate the app.

```
Container(
  width: 150,
  height: 150,
  alignment: Alignment.center,
  child: _bmi==0.0 ? Text(
    textAlign: TextAlign.center,
    'Enter body weight and height to know your Body Mass Index (BMI)',
    style: TextStyle(fontSize: 12, color: Colors.orange, ),
  ): Text(''),
),
```

Notice that the second Text widget displays text conditionally based on the value of a variable named `_bmi`. In this case, the code displays a message prompting the user to enter their weight and height. Once the user enters the values and the BMI is calculated, `_bmi` will no longer be 0.0, and the empty Text widget will be displayed, effectively hiding the initial message. Save and run your app, you should see the following outputs:



17. Continue to insert two Text widgets. These are to display BMI value and the status.

```
//TODO 10: Insert text widget showing BMI status
const Text(
  'You Body Mass Index (BMI) is :',
),
Text(
  _bmiOutput,
  style: Theme.of(context).textTheme.displaySmall,
),
```

18. Continue to insert two TextField widgets, which are to capture inputs from the user.

```
//TODO 11: Insert TextField and Button
TextField(
  controller: _weightCtrl,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Enter weight (kg)',
  ),
),
TextField(
  controller: _heightCtrl,
  keyboardType: TextInputType.number,
  decoration: InputDecoration(
    labelText: 'Enter height (meter)',
  ),
),
```

Setting the **keyboardType** to **TextInputType.number** within the TextField widget restricts the user input to numbers.

19. Insert an Expanded widget that takes up empty screen spaces and pushes the subsequent widget to the bottom of the screen.

```
//TODO 12: Insert an Expanded widget to take up empty screen spaces  
Expanded(child: SizedBox(height: double.infinity,)),
```

20. Finally, insert a Row widget, which has two buttons. The Row widget is used to arrange the buttons horizontally.

```
//TODO 13: Insert reset and calculate buttons  
Row(  
  mainAxisAlignment: MainAxisAlignment.center,  
  children: [  
    ElevatedButton(  
      onPressed: _resetScreen, child: Text('Reset')),  
    ElevatedButton(  
      onPressed: _calculateBMI, child: Text('Calculate')),  
  ],  
)
```

Save and run your app.



TODO: Enhance the app by including input validation. Implement range field validation, ensuring that input falls within an acceptable range of values (e.g. age between 0 and 100).